

System Architecture: LLM Firm Laboratory

Overview

The simulator is a multi-agent system where each agent is an independent application with its own LLM, private database, and analytical tools. A central

Orchestrator coordinates turns, enforces accounting rules, and maintains the canonical shared state. Agents connect to the orchestrator via HTTP.

```
ORCHESTRATOR (one machine)
|   Accounting, validation, turn control, shared database
|
+--- HTTP POST/JSON ---> ENVIRONMENT agent (own machine or shared)
|                           LLM + private DB + analysis tools
|
+--- HTTP POST/JSON ---> FIRM agents x5 (each on own machine or shared)
|                           Each: LLM + private DB + analysis tools
|
+--- HTTP POST/JSON ---> FINANCIAL INSTITUTION agents x3 (each on own machine)
|                           Each: LLM + private DB + analysis tools
```

Total: 11 participants (1 orchestrator + 1 environment + 5 firms + 4 financial agents)

The 4 financial agents are: Equity Market (prices equity, decides subscriptions; acts as PE/VC for private firms, stock market for public firms), Investment Bank (advisory, IPO structuring, M&A advisory), Commercial Bank (revolving credit), Credit Fund (term debt). See doc 14 for simulation modes (public vs. private start) and complexity toggles that control which features are active.

Each agent is NOT a thin LLM wrapper. It is a small autonomous application that:

- Maintains a persistent private SQLite database (decisions, reasoning, observations)
- Runs statistical analysis on its own history and the shared database
- Executes multi-step reasoning (analyze -> reflect -> decide) with 3+ LLM calls per turn
- Produces justified decisions with full reasoning traces stored for future reference

What an Agent Actually Is

Agent as Application (not just an API call)

Each agent is a self-contained Python application:

```
agent_machine/
  agent/
    server.py           # FastAPI -- HTTP interface to orchestrator
    brain.py            # Multi-step reasoning engine (3+ LLM calls per turn)
    analyst.py          # Statistical analysis tools (pandas, numpy, scipy)
    memory.py           # SQLite memory management
    tools.py            # Analysis functions the reasoning engine can invoke
    schemas.py          # Input/output JSON validation
  data/
```

```

shared/           # Read-only mirror of shared data
  past_simulations/ # Compustat panels from prior runs
  world_docs/      # The 7 world-building documents
private/          # Agent-specific, grows over the run
  memory.db        # SQLite: decisions, reasoning, observations, reflections
  analytics_cache/ # Cached analysis results
prompts/
  system_prompt.md # Role-specific system prompt
  analysis_prompt.md # Template for the analysis step
  reflection_prompt.md # Template for the reflection step
  decision_prompt.md # Template for the final decision step
config.yaml       # agent_id, role, model, fingerprint, orchestrator_url
requirements.txt   # fastapi, uvicorn, pandas, numpy, requests, pyyaml
run.sh            # pip install && python agent/server.py

```

The Reasoning Loop (every turn)

When the orchestrator sends a turn prompt, the agent does NOT make one LLM call. It runs a structured reasoning pipeline:

```

1. OBSERVE
  Parse the incoming context from the orchestrator.
  Store all new observations in memory.db.
  |
2. ANALYZE
  Run statistical queries on private data + shared database.
  - Trend analysis on own financials (revenue growth, margin trajectory)
  - Competitor analysis from public data (price movements, share shifts)
  - Historical comparison from past simulations (similar firms, similar situations)
  - Portfolio analysis (for financial institutions: exposure, loss rates)
  Results are structured data, not LLM output.
  |
3. REFLECT (LLM call #1)
  Present the analysis to the LLM. Ask: "What is the situation?
  What are the key dynamics? What risks and opportunities do you see?"
  The LLM interprets the numbers, identifies patterns, forms a view.
  Store the reflection in memory.db.
  |
4. STRATEGIZE (LLM call #2)
  Present the reflection + constraints (budget, capacity, etc.).
  Ask: "What are your strategic options? Evaluate 2-3 alternatives
  with pros and cons."
  The LLM generates and evaluates strategic options.
  |
5. DECIDE (LLM call #3)
  Present the strategic options. Ask: "Choose your actions for this
  quarter. Provide specific numbers and justify each choice."
  The LLM produces the final structured decision JSON.
  |
6. STORE
  Save the full reasoning trace to memory.db:
  - Analysis results

```

- Reflection text
 - Strategic options considered
 - Final decision and justification
- This becomes part of the agent's memory for future turns.

This means each agent makes 3+ LLM calls per turn, plus database queries and statistical computations. A quarter with 9 agents could involve 25-30+ LLM calls total.

Agent Memory (SQLite, persistent)

Each agent maintains a private SQLite database with tables:

```
-- What I decided and why
CREATE TABLE decisions (
    quarter INTEGER,
    phase TEXT,          -- 'ipo', 'quarterly', 'pricing', etc.
    decision_json TEXT,   -- the structured decision
    reasoning TEXT,       -- full reasoning trace
    analysis_summary TEXT, -- key statistics that informed the decision
    created_at TIMESTAMP
);

-- What I observed from the world
CREATE TABLE observations (
    quarter INTEGER,
    source TEXT,          -- 'orchestrator', 'public_info', 'environment_narrative'
    content_json TEXT,    -- structured observation data
    created_at TIMESTAMP
);

-- Periodic self-assessments
CREATE TABLE reflections (
    quarter INTEGER,
    reflection TEXT,      -- LLM-generated strategic assessment
    key_metrics TEXT,     -- JSON of key performance indicators
    strategic_stance TEXT, -- current strategic direction
    created_at TIMESTAMP
);

-- Analysis results (cached for prompt inclusion)
CREATE TABLE analyses (
    quarter INTEGER,
    analysis_type TEXT,   -- 'trend', 'competitor', 'historical_comp', 'portfolio'
    result_json TEXT,
    created_at TIMESTAMP
);
```

Agent Analytical Tools

Each agent has access to Python functions for data analysis:

```
# Available to all agents
def trend_analysis(series: list[float], periods: int) -> dict:
    """Growth rates, moving averages, trend direction."""
```

```

def summary_statistics(series: list[float]) -> dict:
    """Mean, median, std, min, max, quartiles."""

def correlation_analysis(x: list[float], y: list[float]) -> dict:
    """Pearson/Spearman correlation, simple regression."""

# Available to firms
def margin_analysis(revenue: list, cogs: list, opex: list) -> dict:
    """Gross, operating, net margins over time."""

def rd_efficiency(rd_spend: list, capability_gain: list) -> dict:
    """Dollars per unit of capability improvement."""

def cash_runway(cash: float, burn_rate: float, credit_available: float) -> dict:
    """Quarters of cash remaining at current burn rate."""

# Available to financial institutions
def portfolio_exposure(loans: dict, equity: dict) -> dict:
    """Concentration, total exposure, loss reserves."""

def credit_metrics(firm_data: dict) -> dict:
    """Leverage, coverage ratios, liquidity ratios."""

def historical_default_rates(past_sims: pd.DataFrame, filters: dict) -> dict:
    """Default rates for similar firms in past simulations."""

# Available to environment
def demand_model_baseline(prices: list, qualities: list, macro: dict) -> dict:
    """Multinomial logit prediction for calibration."""

def event_probability_check(event_type: str, quarter: int) -> dict:
    """Reference probabilities from world docs for consistency."""

```

Orchestrator

The orchestrator is the **ONLY** participant that is not LLM-powered. It is deterministic Python code running on the main machine.

Responsibilities

- Turn management: Sequences the 9 phases of each quarter
- Information control: Decides what each agent sees (configurable per regime)
- Accounting postings: Translates decisions + outcomes into journal entries
- Feasibility clamping: Enforces spending limits based on available resources
- Settlement and solvency: Draws revolvers, checks cash, triggers defaults
- Bankruptcy processing: Auctions, waterfall, entry/exit
- Validation: Hard invariant checks after every quarter
- Canonical database: Maintains the Compustat panel and debrief CSV

- Shared data sync: Pushes updated shared data to agents between quarters
- Measurement regime: Applies the configured accounting rules (see doc 06)

What the Orchestrator Does NOT Do

- Does not make strategic decisions (no LLM)
- Does not set prices, allocate demand, or underwrite credit
- Does not interpret or second-guess agent reasoning
- Does not modify agent decisions beyond hard feasibility clamping

Communication Protocol

Simple HTTP + JSON

```

Orchestrator                                Agent
|                                             |
| --- POST /turn {context} ----->|
|                                     | (agent runs 3+ LLM calls internally)
|                                     | (agent queries its databases)
|                                     | (agent stores reasoning)
|<-- 200 OK {decision} -----|
|                                     |

```

Endpoints per agent server:

- POST /turn -- receive context, return decision (the main loop)
- POST /sync -- receive updated shared data (between quarters)
- GET /health -- liveness check
- GET /memory/summary -- retrieve agent's memory summary (for diagnostics)
- POST /reset -- reset state for new run or new incarnation

Message Sizes

Message	Typical Size	Notes
Turn context (orchestrator -> agen	20-100 KB	Grows with memory inclusion
Decision response (agent -> orches	2-10 KB	Structured JSON + reasoning text
Shared data sync	100 KB - 5 MB	Compustat panel grows over run

Timeouts and Failures

- Agent response timeout: 180 seconds (agents do 3+ LLM calls internally)
- On timeout: one retry, then use fallback decision (see doc 04)
- On invalid response: one retry with error feedback, then fallback
- Agent health check: before each quarter, ping all agents

Deployment Models

Model A: Development (single machine)

All agents run as separate processes on one machine, communicating via localhost. Good for development and testing.

```
One machine:
Orchestrator (port 8800)
Environment agent (port 8801) -> local Ollama
Firm agents (ports 8802-8806) -> local Ollama
Financial agents (ports 8807-8809) -> local Ollama
```

Downside: 10 agents * 3 LLM calls per turn = 30 sequential calls through one Ollama instance. Slow but functional.

Model B: Production (distributed)

Each agent on its own machine with its own Ollama or API access. Agents within the same phase run in parallel.

```
Machine 0: Orchestrator
Machine 1: Environment agent + Ollama (strongest model -- hardest job)
Machines 2-6: Firm agents + Ollama each
Machines 7-9: Financial institution agents + Ollama each
```

Agents in the same phase (e.g., all 5 firms in Phase 3) are called in parallel. Typical quarter: 4 serial phases with LLM calls, each with parallel agents. Wall-clock time per quarter: ~4 max(agent response times) rather than 30 avg.

Model C: Hybrid

Group agents on fewer machines (e.g., 3 machines running 3 agents each). The orchestrator sends parallel requests; each machine's Ollama handles its local agents sequentially.

Deployment Package

Each agent is deployed as a zip:

```
firm_agent_v1.zip (~2-5 MB)
requirements.txt
setup.sh          # pip install, init DB, verify ollama
run.sh           # start server
config.yaml      # customize: agent_id, fingerprint, orchestrator_url, model, port
agent/           # Python source
data/shared/     # world docs, past simulation summaries
data/private/    # initialized empty
prompts/         # role-specific prompt templates
```

The recipient runs: unzip, bash setup.sh, edit config.yaml, bash run.sh.

Configurable Regimes (see doc 06 for full details)

Two key axes of configuration control experimental variation:

1. Information Regime

Controls WHAT each agent type can see. Configured in config.yaml:

```
information_regime: "baseline"
# Alternatives: "full_transparency", "minimal_disclosure", "asymmetric", custom
```

Examples:

- "baseline": firms see public financials + macro; institutions see statements
- "full_transparency": everyone sees everything (benchmark)

- "minimal_disclosure": firms see only revenue + net income of competitors
- Custom: per-agent-type specification of visible fields

2. Measurement Regime

Controls HOW financial statements are constructed. Configured in config.yaml:

```
measurement_regime: "baseline_gaap"
# Alternatives: "rd_capitalization", "fair_value_assets", "cash_basis", custom
```

Examples:

- "baseline_gaap": R&D expensed, historical cost, accrual basis
- "rd_capitalization": 60% of R&D capitalized and amortized
- "fair_value_assets": PPE marked to noisy fair value each quarter

Both regimes are parameters of the orchestrator, not of the agents. The orchestrator applies the regime when constructing the context sent to each agent. Agents just see "their" information -- they don't know what others see.

Turn Sequence (Summary)

```
PHASE 1: Shock Generation ..... [Orchestrator] deterministic
PHASE 2: IPO for new entrants ..... [Firm -> IBank -> CBank -> CFund] LLM calls
PHASE 3: Firm decisions ..... [5 Firms in parallel] LLM calls (3+ each)
PHASE 4: Feasibility clamping ..... [Orchestrator] deterministic
PHASE 5: Market resolution ..... [Environment] LLM calls (3+)
PHASE 6: Accounting postings ..... [Orchestrator] deterministic
PHASE 7: Financial inst. decisions .. [3 FIs in parallel] LLM calls (3+ each)
PHASE 8: Settlement + defaults ..... [Orchestrator] deterministic
PHASE 9: Record-keeping + sync ..... [Orchestrator] deterministic
```

See doc 04 for detailed message formats and JSON schemas. See doc 07 for failure handling (firm bankruptcy, bank failure, equity valuation).